

Department of Computing and Mathematics

ASSESSMENT BRIEF

Module Code and Title:	6G5Z0059 Software Design and Architecture
Assessment Set By:	Steve.Baker@mmu.ac.uk
Assessment ID:	1CWK100
Assessment Weighting:	100%
Assessment Title:	1CWK100 Software Product Design
Version	1.1 (corrected the State Machine Implementation scenario output)
Type:	Individual
Hand-In Deadline:	See Moodle
Hand-In Format and Mechanism:	Submission online via Moodle

Learning outcomes being assessed:**LO1: Design software systems using an industrial standard design process.****LO2: Identify and apply a range of software design patterns.****LO3: Evaluate software architecture and design.**

Note: it is your responsibility to make sure that your work is complete and available for marking by the deadline. Make sure that you have followed the submission instructions carefully, and your work is submitted in the correct format, using the correct hand-in mechanism (e.g., Moodle upload). If submitting via Moodle, you are advised to check your work after upload, to make sure it has uploaded properly. If submitting via OneDrive, ensure that your tutors have access to the work. Do not alter your work after the deadline. You should make at least one full backup copy of your work.

Penalties for late submission

The timeliness of submissions is strictly monitored and enforced.

All coursework has a late submission window of 7 calendar days, but any work submitted within the late window will be capped at 40%, unless you have an agreed extension. Work submitted after the 7-day late window will be capped at zero unless you have an agreed extension. See 'Assessment Mitigation' below for further information on extensions.

Please note that individual tutors are unable to grant any extensions to assessments.

Assessment Mitigation

If there is a valid reason why you are unable to submit your assessment by the deadline you may apply for Assessment Mitigation. There are two types of mitigation you can apply for via the module area on Moodle (in the 'Assessments' block on the right-hand side of the page):

- **Non-evidenced extension:** does **not** require you to submit evidence. It allows you to add a **short** extension to a deadline. This is not available for event-based assessments such as in-class tests, presentations, interviews, etc. You can apply for this extension during the assessment weeks, and the request must be made **before** the submission deadline. For this assessment, the non-evidenced extension is 2 days.
- **Evidenced extension** requires you to provide independent evidence of a situation which has impacted you. Allows you to apply for a longer extension and is available for event-based assessment such as in-class test, presentations, interviews, etc. For event-based assessments, the normal outcome is that the assessment will be deferred to the summer reassessment period.

Further information about Assessment Mitigation is available on the dedicated [Assessments page](#).

Personal Learning Plans

If you have a [Personal Learning Plan \(PLP\)](#) which states you can negotiate an extended deadline, make an appointment to see the [Department's Disability Coordinator](#) to discuss to discuss your needs, and where appropriate agree on a revised submission deadline.

Plagiarism

Plagiarism is the unacknowledged representation of another person's work, or use of their ideas, as one's own. Manchester Metropolitan University takes care to detect plagiarism, employs plagiarism detection software, and imposes severe penalties, as outlined in the [Student Code of Conduct](#) and [Academic Misconduct Policy](#). Poor referencing or submitting the wrong assignment may still be treated as plagiarism. If in doubt, seek advice from your tutor.

As part of a plagiarism check, you may be asked to attend a meeting with the Module Leader, or another member of the module delivery team, where you will be asked to explain your work (e.g. explain the code in a programming assignment). If you are called to one of these meetings, it is very important that you attend.

Use of generative AI

The use of generative AI is permitted in this assessment, so long as it is used in accordance with the instructions provided in the 'Are you allowed to use AI in assessments?' section of the [AI Literacy Rise Study Pack](#). All submitted work must be your own original content.

If you are unable to upload your work to Moodle

If you have problems submitting your work through Moodle, you can send your work to the Assessment Management Team using the [Contingency Submission Form](#). Assessment Management will then forward your work to the appropriate person for marking. If you use this submission method, your work must be sent **before the published deadline**, or it will be logged as a late submission. Alternatively, you can save your work into a single zip folder then upload the zip folder to your university OneDrive and submit a Word document to Moodle which includes a link to the folder. **It is your responsibility to make sure you share the OneDrive folder with the Module Leader, or it will not be possible to mark your work.**

Assessment Regulations

For further information see the [Undergraduate Assessment Regulations](#) on the [Assessments and Results information pages](#)

Formative Feedback:	<i>Lab sessions will be dedicated to assessment support and to provide verbal formative feedback. Additional support can be obtained in Tutor office hours (check Moodle for times).</i>
Summative Feedback:	<i>Written feedback will be provided after submission and before the individual feedback date shown on Moodle. This</i>

	<i>feedback will be written with reference to the summative marking scheme.</i>
--	---

Introduction to the Assessment Task

The assessment task is an opportunity to demonstrate your software design skills and knowledge of software design patterns and software architecture. The task is to design a small software product using the principles, patterns and processes covered in the taught materials (including the module textbook and background reading) and lab exercises of the Software Design and Architecture module.

The module is 100% coursework based, and this assessment component is weighted at 100% of the module marks. This is an individual activity.

Marks are awarded for meeting the functional and architectural requirements (outlined below) and for designing and writing the code in an object-oriented manner using common design patterns. There are additional marks available for implementing advanced features.

The deliverables (the things you need to produce and submit via Moodle) are:

- A Java **console** application that plays the game and meets some or all the functional specification below. A graphical user interface is **NOT** required.
- A **readme** document explaining your design.

The Submission

Your submission should be in the form of a ZIP file containing:

- The complete source code for your submission as an IntelliJ project.
- A readme file written in Markdown.

Please name your submission file as

**Firstname_Lastname_StudentIDNumber.zip (Example:
Steve_Williams_12345678.zip).**

Moodle submission inboxes cannot accept files larger than 100 MB. It is unlikely that you will exceed this for this assessment. Should you need to submit files exceeding this size you are required to upload your project to your university-supplied OneDrive account and submit a link as part of the package you submit to Moodle (e.g., pasted into a text file). Prior to submission via Moodle, please ensure that you archive your project folders into a zip file and upload them to your OneDrive account.

The Assessment Project

You are simulating a prototype physical board game called “Simple Frustration”. There is a demo in Lecture 1 that shows an example simulation in action. The game rules are discussed in the Appendix.

A simulation is a model of the board game which:

- Starts a game (setting up the board and player pieces)
- Takes an input of a roll of one die or two dice and automatically alternates between players.
- Tracks the position of the players around the board and implements the game rules.
- Outputs the move made by each player.

Your software is to be written as a console application (you do **NOT** need to create a GUI or Web interface).

Your software will need to output progress information and play statistics as play progresses.

The board game designers want to vary elements of the design to understand from the simulation how this affects game play, for example if it makes play too easy or too difficult.

You will also need to produce a README document which explains your design and design choices. See the [Readme](#) section of this document for details.

Functional Specification

The simulation must follow the Simple Frustration game rules (supplied in the Appendix).

The board game designers have asked for the simulation to output information to the console about the progress of a game:

1. Player takes turns to play until the game is won.
2. Each turn, print the current player, the value of the dice roll(s) and the position of the player before and after the turn.
3. Each turn, print the running total of the number of turns taken by the current player.
4. At the end of the game (i.e. the game has been won) print the winning player.
5. At the end of the game (i.e. the game has been won) output the total number of turns taken by all the players.

A game simulation that follows the basic game rules and meets the functional specification is called the **basic game**. See the Appendix for a full explanation of the game rules with examples.

Functional Variations

The board game designers have asked for the simulation to be run with different **variations**.

A simulation should be able to run with any combination of the variations.

Marks are awarded based on number of variations implemented.

- **Single Die.** Player throws a single 6-sided die each turn. Having implemented this variation, the simulation can run with either two dice or a single die.
- **Player must land exactly on the END position to win.** The player needs to throw the exact number to reach END – if the number is too high, the player stays where they are (the player's turn is forfeit). Having implemented this variation, the simulation can run with two different rules for handling the END position.
- **Player stays where they are if they HIT another player.** If a player would **Hit** another player, the player stays where they are (the player's turn is forfeit). For example, if the **Red** player would have landed on a position occupied by the **Blue** player, the Red player stays where they are (as if the turn had not happened). Likewise, if a **Blue** player piece would have landed on a position occupied by **Red**, then the Blue player's turn is forfeit. Having implemented this variation, the simulation can run with two different rules for handling a HIT.

See the Appendix for a full explanation of the variations with examples.

Your readme document should clearly state which variations you have attempted and explain their design.

Advanced Features

If you have met the functional specification and completed the basic game with all the variations, then additional marks can be obtained by implementing advanced features.

Advanced features should only be attempted **once you have met the functional specification and completed the basic game with all the variations**. Implementing advanced features will require additional reading as the concepts and techniques will not necessarily be covered in lectures and labs. The accompanying course textbook will be a good starting point.

- **Large Board with 4 players.** The game can be played with four players (Red, Blue, Green and Yellow) on a larger board having 36 positions on the main board (in the basic game there are 18 positions on the main board) and 6 positions in the tail including the **End** (in the basic game there are 3 positions in the tail including the **End** place). Red **Home** position is 1, the Blue **Home** position is 10, the Green **Home** position is 19 and the Yellow **Home** position is 28. Having implemented this variation, the simulation can run with two different board sizes.
- **Game States.** Use the State pattern or other valid implementation of a state machine to create a game in a **Ready** state, transition to an **In Play** state when in play and transition to a **Game Over** state when there is a winner. We require each state transition to be printed to screen. In the **Game Over** state any additional attempt to play a piece by providing the game with additional dice rolls will result in printing a 'Game Over' message. We require an implementation of a state machine, not just procedural logic, to count as a successful variation.
- **Game Save and Replay.** A game is saved at the end of the game to a file or in-memory data store against an id. The stored game can then be retrieved and the game replayed so that the board game designers can examine the output. There are two ways of doing this, you could store the game configuration (board size, number of players etc) and the sequence of dice rolls and replay the game using your game code, or store the output from the game, retrieved the saved output and display it. You should explain which method you chose and why you chose it

in your README document.

- **Unit Testing.** There is evidence that the design is testable by providing a suite of unit tests that can be executed by the assessor.

See the Appendix for a full explanation of the advanced features with examples.

Your README document should clearly state which advanced features you have attempted and explain their design.

Architectural Specification

A development team in industry will have a coding standard and an architectural standard describing how software should be constructed. We will cover the architectural specification in the latter part of the module.

Your submitted code will need to fulfil the following coding standards

- Code to be written in Java 21 or higher. 24 is the current version at time of writing, but 25 may be released during the module
- Code to follow the Google Java coding guidelines¹.
- Code should make use of design patterns to solve design problems. Your README document should include an explanation of where and why design patterns have been used (naming the design pattern).
- The application uses the Dependency Injection container from Spring Boot Framework.
- Is written in the Clean Architecture style discussed in lectures and labs.

The Clean Architecture style means that your submitted code will need to fulfil the following criteria:

- The domain logic (all the game mechanics) is separated from the technology (such as writing to the screen and saving/retrieving files).
- Correctly places code into domain, usecase, and infrastructure packages according to their responsibilities.
- Package dependencies follow Clean Architecture principles (infrastructure depends on usecase and domain).
- Domain and use case code is testable with no direct dependencies on infrastructure code.
- The application is assembled using Dependency Injection.

¹ <https://google.github.io/styleguide/javaguide.html>

The README Document (Report)

Working in a development team you will often need to explain how your code is organized and works to others in the form of a README document. In the past this would have been in the form of a technical report, but modern software practice is to provide a README document in Markdown. The intention of the README is to enable another developer to pick up and continue your work.

Your README document must be written in **Markdown**, which is a lightweight markup language. Markdown is introduced in lectures and labs. The document file name therefore becomes README.md.

IDEs such as Intelli-J with a Markdown plugin will show a preview of the document.

Git Repository portals such as Github will automatically recognize and display a README.md in the root directory.

For this assessment task your README file should contain the following information:

- The variations and advanced features attempted and their design.
- An explanation of where and why design patterns have been used (naming the design pattern).
- An explanation of where and why SOLID principles have been followed (naming the principle).
- An explanation of how you have applied clean or “ports and adapters” architecture with reference to dependencies
- An evaluation of your implementation.

There is no word count limit- we are asking you to explain your design and architecture, not to document your code, so as a guide 1000-1500 words should be sufficient excluding tables and code examples.

- Use of Markdown formatting to create a visually pleasing document will be necessary for high marks.
- The use of the correct technical terms (see lecture notes and supporting material) will be essential for high marks.

- Higher marks can be obtained by correct use of UML diagrams. These can be provided using Mermaid (covered in lectures and labs).

How to show evidence of your simulation working

No GUI is required. Your application can simply write to the console, using (for example) the Java `System.out.printf` or `System.out.format` methods.

Example output which meets the functional specifications is shown in the Appendix.

How to show your variations and advanced features

If you have implemented variations or advanced features, send output to the console indicating the configuration of the board, the number of players, the game rules and the number of dice used for a simulation run.

Example output which meets the functional specifications is shown in the Appendix.

Support

There is planned time in labs to get support and formative feedback.

For additional support on the assessment, contact Steve Baker

(steve.baker@mmu.ac.uk). Contact details and office hours are on the module page in Moodle.

Suggested Approach and Schedule

The module covers design first and then architecture from around week 7/8. Therefore, you should concentrate on the functional specification and the application of design patterns first and then refactor your code to the end-state architecture after we have covered the architectural specifications.

Do not leave starting this assignment to the end of the module – good design takes time and many iterations, and you will not be making good use of lab sessions for formative feedback.

Ensure you have a working development environment as soon as possible – we set this up in the first week lab.

The essential skills of software design are identifying and creating classes and applying design patterns.

- Classes should be written and laid out using the coding and layout conventions discussed in lectures and labs.
- Code should use object-oriented techniques such as **interface implementation, inheritance, polymorphism, and encapsulation**.
- Classes should conform to the **SOLID (single responsibility, open/closed, Liskov substitution, interface segregation and dependency inversion)** principles.
- Standard **design patterns** used where appropriate.
- The implementation makes minimal use of if/else or switch statements, preferring instead to use abstractions and strategies.

Suggested Schedule

- Week 1** Have your development environment setup. We strongly advise the use of Git to manage your work, so get a Github account and practice basic Git operations.
- Week 2** Start by quickly coding the basic game. At this stage you are not designing, you are just writing some code (you could try and do everything in the single main method if you wish) to explore and understand the problem. Use the scenarios in the Appendix to check your understanding of the game.
- Week 3** Refactor your basic game code into classes. A superior design will require many classes. Take time to identify the classes and assign responsibilities to them using the principles we have discussed in the material.
- Week 4** Code some variations. Having coded an object-oriented version of the basic game first, you will have a good understanding of the game works and therefore will be in a better position for tackling the variations requested by the designers.
- We would suggest taking on each requested variation one at a time. Trying to write an initial version that implements all the variations will be overwhelming, and taking an iterative approach means that you will always have a working version to submit.
- Review the lecture material on handling variations using design patterns.
- Week 5** Continue with the variations. Refactor the code to use patterns where possible. Attempt advanced features if you have got all the variations done.
- Week 6,7,8,9** Review and get feedback on your design at the lab sessions. Refactor the code based on feedback. Start the README document that documents your design.
- Week 10** Setup your Clean Architecture framework

Week 11 Review and get feedback on your architecture at the lab sessions.
Refactor the code based on feedback. Complete the parts of README
document that documents your architecture.

Hints and Tips for designing and writing the Functional Code

Review the coding conventions covered in week 1. Ensure you are writing well formatted code that conforms to the coding conventions.

There is no “right” design. We are looking for sensible use of patterns (too much pattern usage better than not enough in this case) and a good assignment of functionality to well named classes.

It is unlikely you will get the design right first time – an iterative approach will be more successful: write a version, take a critical look (analyse) your design using the knowledge gained from the module, and reflect on where it can be improved, decide what you are going to change and produce another version.

Use of Git is highly recommended. Create a Git branch for each design change – not all design experiments are successful, and being able to revert to your previous version can save a lot of time.

You are going to need both a random number generator for simulating the dice, but for testing and development purposes this will become very frustrating (no pun intended). Therefore, we would recommend that a random dice roll will need to be swappable with a dice role generator that creates a fixed sequence of rolls, so that you can create a fixed sequence of dice rolls for testing purposes. The examples in the Appendix show relevant dice sequences.

Unit testing using JUnit was introduced in Advanced Programming. Although not required for this assessment, you may find it helpful to write unit tests for your individual classes. If you do produce unit tests **include them in your submission**.

Ensure you have a code version that compiles and executes that you could hand in at the end every week.

Hints and Tips for meeting the Architecture Specification

Using the website <https://start.spring.io> create a starter project using the following settings

- Project: Maven
- Language: Java
- Sprint Boot: Choose latest stable (at time of writing this was 3.5.5)
- Project Metadata – Group: uk.ac.mmu
- Project Metadata – Artifact: game
- No dependencies are required.

Generate a project which will provide the basis for your solution. We will also provide an example in the labs.

Clean Architecture (Ports and Adaptors)

An example of a clean architecture using ports and adaptors will be provided in labs.

Use Cases

You will need to implement the following use cases for a complete architecture.

Play – given a configuration (board size, number of players, variations) play the game to completion and save the game for later replay to a data store.

Replay – given the id of a game, retrieve it from the data store and replay it.

Using JSON serialisation

If you decide to write your game data to a file using JSON serialisation, one standard library is Jackson.

The simplest method of bringing in Jackson is to use spring-boot-starter-json.

Change the dependency in the pom.xml generated by start.spring.io from **spring-boot-starter** to **spring-boot-starter-json**

```
<dependency>

    <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-json</artifactId>  
</dependency>
```

If you change the pom.xml file manually in the texteditor, you need to synchronize the changes.

1. In the Maven tool window, right-click a linked project.
2. From the context menu, select Sync Project the Sync project icon.

See <https://www.jetbrains.com/help/idea/delegate-build-and-run-actions-to-maven.html>

Follow instructions in <https://github.com/FasterXML/jackson-databind> for how to serialize and deserialize objects to JSON.

Locating your file

You can store your game files anywhere, but usually these kinds of files would be stored in a user's private area. Unfortunately, where this is depends on operating system – Windows, Linux and MacOS will all use a different location on disk for user files.

Java has a useful properties collection in the System class that abstracts away some of the operating system specifics, specifically the "user.home" property which provides the user's home directory as a string in an OS independent way.

```
String home = System.getProperty("user.home");
```

See <https://docs.oracle.com/javase/tutorial/essential/environment/sysprop.html>

Hints and Tips for writing the README document

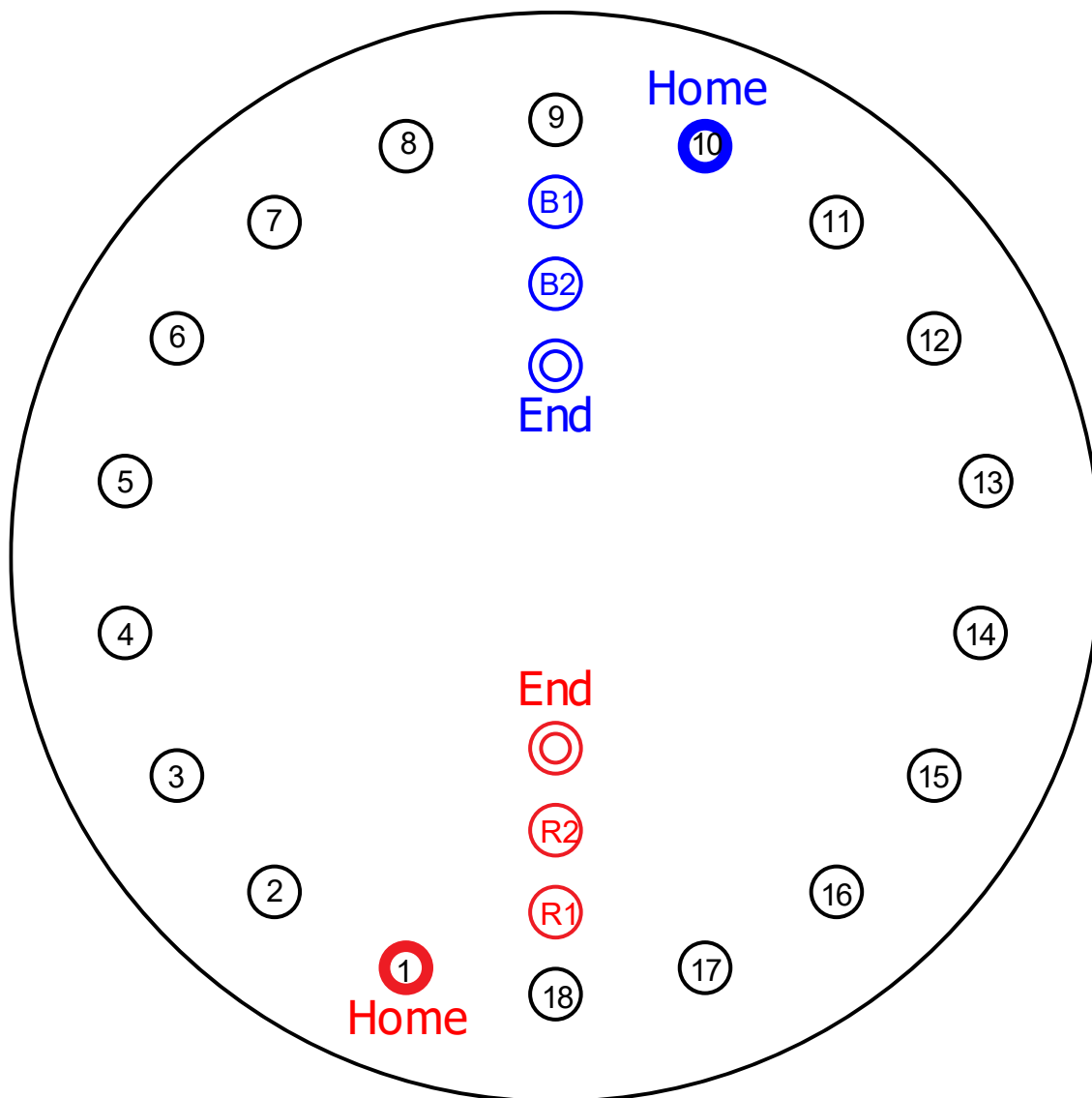
Because you will go through different design options, it is OK to write the README document at the end but make notes as you create the code so that you remember your design decisions. We suggest that you write up the design of the game first and then write up your explanation of how you have applied clean or “ports and adapters” architecture and an evaluation of your implementation.

Writing a good README document takes effort, so don’t leave this to the last minute.

Mark Scheme

Criteria and % of Overall Mark	Clear Fail 0-29%	Marginal Fail 30-39%	Pass 40-49%	2:2 50-59%	2:1 60-69%	1 70-100%
Functionality provided by the submitted code (40 marks) <i>Code submitted without an accompanying README.md document will NOT be marked.</i>	Code does not compile or does not run, or minimal functionality.	Basic game attempted with minimal success.	Basic game attempted with some success; some variations attempted.	Basic game functions successfully, some variations attempted or implemented.	Basic game functions successfully, multiple variations attempted or implemented.	Basic game functions successfully and all variations implemented. Product has high quality input validation and high-quality outputs. Advanced features attempted or implemented for the highest marks.
Quality of Submitted code (40 marks) <i>Code submitted without an accompanying README.md document will NOT be marked.</i>	Code does not compile or does not run or minimal functionality or architecture	Poor quality code that has no or little object-orientation, does not follow SOLID principles. Code does not conform to the architectural specification.	Code has some object-orientation and follows some SOLID principles. Code conforms to some elements of the architectural specification.	Code is generally object orientated and generally follows SOLID principles. Code generally conforms to the architectural specification.	Code is completely object-oriented, follows SOLID principles making use of design patterns where appropriate. Code conforms to the architectural specification.	Code is completely object-oriented, expertly follows SOLID principles and makes extensive use of design patterns where appropriate. Code conforms to the architectural specifications in an exemplary manner.
Quality of README Document. (20 marks) <i>A README document submitted without accompanying code will NOT be marked.</i>	No README or basic README that documents some of the design to some extent.	A README that documents some or of the design but with an insufficient level of detail or accuracy.	A README that documents the design acceptably but doesn't sufficiently explain the software design techniques used.	A README that documents the design well, with some appropriate discussion of software design techniques used to improve quality using some correct terminology and possibly including diagrams where appropriate.	A README that comprehensively documents the design with detailed and well-informed discussion of software design techniques used to improve quality, using generally accurate terminology supported with some appropriate diagrams.	A README is included that rigorously documents the design, with comprehensive discussion of well-applied software design techniques used to improve quality using accurate terminology supported by good use of appropriate diagrams.

APPENDIX: SIMPLE FRUSTRATION GAME RULES

**Basic Game Rules**

This is a game for two players. **Red** and **Blue**.

- The board has 18 positions and 2 **Tails** (one for each player) with 3 positions each.
- Each player starts with a player piece in their **Home** position. To win the game, a player must get their player piece to their **End** position first.
- Players take turns to roll 2 x 6-sided dice.
- They move *clockwise* around the board based on the number of the dice roll.

- The **Red** player moves clockwise through positions 1 (**Home**), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (**End**).
- The **Blue** player moves clockwise through positions 10 (**Home**), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (**End**)
- If a player piece lands on a position occupied by another player piece, then this is called a **Hit**. The basic game allows for multiple players to occupy the same position.
- The game continues until either Red or Blue have reached their **End** position. This could be done by throwing a dice roll that lands the player on or beyond the **End** position. Throwing a dice roll that would move the player beyond the **End** position is called an **Overshoot**.

Basic Game Scenarios

All these scenarios use the basic game (i.e. before any variations).

Basic Game Scenario 1

Blue wins.

Board length Board positions=18 Tail positions=3

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (End)

HITS are ignored, multiple players can occupy the same position

Player can land on or beyond the END position to win

Dice: Sequence of dice rolls (12,12,7,8)

Red turn 1 rolls 12

Red moves from Home (Position 1) to Position 13

Blue turn 1 rolls 12

Blue moves from Home (Position 10) to Position 4

Red turn 2 rolls 7

Red moves from Position 13 to Tail Position R2

Blue turn 2 rolls 8

Blue moves from Position 4 to End (Position B3)

Blue wins in 2 turns!

Total turns 4

Another example

Dice: Sequence of dice rolls {12,12,6,6,2}

Red turn 1 rolls 12

Red moves from Home (Position 1) to Position 13

Blue turn 1 rolls 12

Blue moves from Home (Position 10) to Position 4

Red turn 2 rolls 6

Red moves from Position 13 to Tail Position R1

Blue turn 2 rolls 6

Blue moves from Position 4 to Tail Position B1

Red turn 3 rolls 2

Red moves from Tail Position R1 to End (Position R3)

Red wins in 3 turns!

Total turns 5

Basic Game Scenario 2

Red wins, HITs are ignored.

Board length Board positions=18 Tail positions=3

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (End)

HITS are ignored, multiple players can occupy the same position

Player can land on or beyond the END position to win

Dice: Sequence of dice rolls (8,2,3,4,9)

Red turn 1 rolls 8

Red moves from Home (Position 1) to Position 9

Blue turn 1 rolls 2

Blue moves from Home (Position 10) to Position 12

Red turn 2 rolls 3

Blue Position 12 hit!

Red moves from Position 9 to Position 12

Blue turn 2 rolls 4

Blue moves from Position 12 to Position 16

Red turn 3 rolls 9

Red moves from Position 12 to End (Position R3)

Red wins in 3 turns!

Total turns 5

Basic Game Scenario 3

Blue wins even though it overshot the END position.

Board length Board positions=18 Tail positions=3

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (End)

HITS are ignored, multiple players can occupy the same position

Player can land on or beyond the END position to win

Dice: Sequence of dice rolls (12,12,7,11)

Red turn 1 rolls 12

Red moves from Home (Position 1) to Position 13

Blue turn 1 rolls 12

Blue moves from Home (Position 10) to Position 4

Red turn 2 rolls 7

Red moves from Position 13 to Tail Position R2

Blue turn 2 rolls 11

Blue overshoots!

Blue moves from Position 4 to End (Position B3)

Blue wins in 2 turns!

Total turns 4

Variations

Examples showing the different variations.

Basic game rules, single die

Blue wins using a single die.

Board length Board positions=18 Tail positions=3

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (End)

HITS are ignored, multiple players can occupy the same position

Player can land on or beyond the END position to win

Dice: Sequence of dice rolls (6,6,6,6,3,4,3,4)

Red turn 1 rolls 6

Red moves from Home (Position 1) to Position 7

Blue turn 1 rolls 6

Blue moves from Home (Position 10) to Position 16

Red turn 2 rolls 6

Red moves from Position 7 to Position 13

Blue turn 2 rolls 6

Blue moves from Position 16 to Position 4

Red turn 3 rolls 3

Red moves from Position 13 to Position 16

Blue turn 3 rolls 4

Blue moves from Position 4 to Position 8

Red turn 4 rolls 3

Red moves from Position 16 to Tail Position R1

Blue turn 4 rolls 4

Blue moves from Position 8 to End (Position B3)

Blue wins in 4 turns!

Total turns 8

Player must land exactly on the END position to win

Red wins, both Red and Blue do not get the exact throw to land on their END positions, so stay in place (turns are forfeit).

Board length Board positions=18 Tail positions=3

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (End)

Player's turn is forfeit if the player would HIT another player

Player must land exactly on the END position to win else the turn is forfeited

Dice: Sequence of dice rolls (12,12,12,9,8)

Red turn 1 rolls 12

Red moves from Home (Position 1) to Position 13

Blue turn 1 rolls 12

Blue moves from Home (Position 10) to Position 4

Red turn 2 rolls 12

Red overshoots!

Red remains at Position 13

Blue turn 2 rolls 9

Blue overshoots!

Blue remains at Position 4

Red turn 3 rolls 8

Red moves from Position 13 to End (Position R3)

Red wins in 3 turns!

Total turns 5

Player stays in place on HIT

Blue wins, Red hits Blue and Red stays in place (Red turn is forfeit).

Board length Board positions=18 Tail positions=3

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (End)

Player's turn is forfeit if the player would HIT another player

Player must land exactly on the END position to win else the turn is forfeited

Dice: Sequence of dice rolls (8,2,3,12,9,6)

Red turn 1 rolls 8

Red moves from Home (Position 1) to Position 9

Blue turn 1 rolls 2

Blue moves from Home (Position 10) to Position 12

Red turn 2 rolls 3

Blue Position 12 hit!

Red remains at Position 9

Blue turn 2 rolls 12

Blue moves from Position 12 to Position 6

Red turn 3 rolls 9

Red moves from Position 9 to Position 18

Blue turn 3 rolls 6

Blue moves from Position 6 to End (Position B3)

Blue wins in 3 turns!

Total turns 6

Advanced Features

Examples demonstrating the advanced features.

Large Board, 4 Players

4 Players, large board, basic rules.

Board length Board positions=36 Tail positions=6

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, R1, R2, R3, R4, R5, R6 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3, B4, B5, B6 (End)

Green 19 (Home), 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, G1, G2, G3, G4, G5, G6 (End)

Yellow 28 (Home), 29, 30, 31, 32, 33, 34, 35, 36, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, Y1, Y2, Y3, Y4, Y5, Y6 (End)

HITS are ignored, multiple players can occupy the same position

Player can land on or beyond the END position to win

Dice: Sequence of dice rolls {7,3,8,5,7,6,8,7,6,8,2,4,4,8,5,7,8,3,9,9,7,5,7,9}

Red turn 1 rolls 7

Red moves from Home (Position 1) to Position 8

Blue turn 1 rolls 3

Blue moves from Home (Position 10) to Position 13

Green turn 1 rolls 8

Green moves from Home (Position 19) to Position 27

Yellow turn 1 rolls 5

Yellow moves from Home (Position 28) to Position 33

Red turn 2 rolls 7

Red moves from Position 8 to Position 15

Blue turn 2 rolls 6

Blue moves from Position 13 to Position 19

Green turn 2 rolls 8

Green moves from Position 27 to Position 35

Yellow turn 2 rolls 7

Yellow moves from Position 33 to Position 4

Red turn 3 rolls 6

Red moves from Position 15 to Position 21

Blue turn 3 rolls 8

Blue moves from Position 19 to Position 27

Green turn 3 rolls 2

Green moves from Position 35 to Position 1

Yellow turn 3 rolls 4

Yellow moves from Position 4 to Position 8

Red turn 4 rolls 4

Red moves from Position 21 to Position 25

Blue turn 4 rolls 8

Blue moves from Position 27 to Position 35

Green turn 4 rolls 5

Green moves from Position 1 to Position 6

Yellow turn 4 rolls 7

Yellow moves from Position 8 to Position 15

Red turn 5 rolls 8

Red moves from Position 25 to Position 33

Blue turn 5 rolls 3

Blue moves from Position 35 to Position 2

Green turn 5 rolls 9

Yellow Position 15 hit!

Green moves from Position 6 to Position 15

Yellow turn 5 rolls 9

Yellow moves from Position 15 to Position 24

Red turn 6 rolls 7

Red moves from Position 33 to Tail Position R4

Blue turn 6 rolls 5

Blue moves from Position 2 to Position 7

Green turn 6 rolls 7

Green moves from Position 15 to Tail Position G4

Yellow turn 6 rolls 9

Yellow moves from Position 24 to End (Position Y6)

Yellow wins in 6 turns!

Total turns 24

The same large board can be played with the variation rules:

Large board, 4 Players, variation rules.

Board length Board positions=36 Tail positions=6

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, R1, R2, R3, R4, R5, R6 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3, B4, B5, B6 (End)

Green 19 (Home), 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, G1, G2, G3, G4, G5, G6 (End)

Yellow 28 (Home), 29, 30, 31, 32, 33, 34, 35, 36, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, Y1, Y2, Y3, Y4, Y5, Y6 (End)

Player's turn is forfeit if the player would HIT another player

Player must land exactly on the END position to win else the turn is forfeited

Dice: Sequence of dice rolls {11,11,8,10,10,7,2,4,6,8,4,9,9,10,7,11,10,8,5,7}

Red turn 1 rolls 11

Red moves from Home (Position 1) to Position 12

Blue turn 1 rolls 11

Blue moves from Home (Position 10) to Position 21

Green turn 1 rolls 8

Green moves from Home (Position 19) to Position 27

Yellow turn 1 rolls 10

Yellow moves from Home (Position 28) to Position 2

Red turn 2 rolls 10

Red moves from Position 12 to Position 22

Blue turn 2 rolls 7

Blue moves from Position 21 to Position 28

Green turn 2 rolls 2

Green moves from Position 27 to Position 29

Yellow turn 2 rolls 4

Yellow moves from Position 2 to Position 6

Red turn 3 rolls 6

Blue Position 28 hit!

Red remains at Position 22

Blue turn 3 rolls 8

Blue moves from Position 28 to Position 36

Green turn 3 rolls 4

Green moves from Position 29 to Position 33

Yellow turn 3 rolls 9

Yellow moves from Position 6 to Position 15

Red turn 4 rolls 9

Red moves from Position 22 to Position 31

Blue turn 4 rolls 10

Blue moves from Position 36 to Tail Position B1

Green turn 4 rolls 7

Green moves from Position 33 to Position 4

Yellow turn 4 rolls 11

Yellow moves from Position 15 to Position 26

Red turn 5 rolls 10

```
Red moves from Position 31 to Tail Position R5

Blue turn 5 rolls 8

Blue overshoots!

Blue remains at Tail Position B1

Green turn 5 rolls 5

Green moves from Position 4 to Position 9

Yellow turn 5 rolls 7

Yellow moves from Position 26 to End (Position Y6)

Yellow wins in 5 turns!

Total turns 20
```

State Machine Implementation

The state machine should move between states, with each state transition printed to the console. When in the **Game Over** state additional dice rolls have no effect other than to result in a console output.

For example.

Blue wins in 2 turns (4 in total), show Game Over behaviour

Board length Board positions=18 Tail positions=3

Red 1 (Home), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, R1, R2, R3 (End)

Blue 10 (Home), 11, 12, 13, 14, 15, 16, 17, 18, 1, 2, 3, 4, 5, 6, 7, 8, 9, B1, B2, B3 (End)

HITS are ignored, multiple players can occupy the same position

Player can land on or beyond the END position to win

Dice: Sequence of dice rolls {12,12,7,8,12,12}

Should output (note the 2 additional game over messages from the last two dice rolls)

```
Game state Ready -> InPlay
```


Red turn 1 rolls 12

Red moves from Home (Position 1) to Position 13

Blue turn 1 rolls 12

Blue moves from Home (Position 10) to Position 4

Red turn 2 rolls 7

Red moves from Position 13 to Tail Position R2

Blue turn 2 rolls 8

Blue moves from Position 4 to End (Position B3)

Blue wins in 2 turns!

Total turns 4

Game state InPlay -> GameOver

Game over

Game over

Appendix: Assessment Mapping to KSBs

Software Design and Architecture	
Core Knowledge	Whether requires evidence for Professional Discussion
A range of digital technology solution development techniques and tools. (K5)	
The approaches and techniques used throughout the digital and technology solution lifecycle and their applicability to an organisation's standards and pre-existing tools. (K6)	Y
Techniques of robust research and evaluation for the justification of digital and technology solutions. (K18)	
Core Skills	
Analyse a business problem to identify the role of digital and technology solutions. (S1)	
Initiate, design, code, test and debug a software component for a digital and technology solution. (S4)	Y
Apply relevant standard processes, methods, techniques and tools. For example, ISO Standards, Waterfall, Agile in a digital and technology solution project. (S5)	
Software Engineering Knowledge	
How to operate at all stages of the software development life cycle and how each stage is applied in a range of contexts. For example, requirements analysis, design, development, testing, implementation. (K21-SEK1)	Y
Principles of a range of development techniques, for each stage of the software development cycle that produce artefacts and the contexts in which they can be applied. For example UML, unit testing, programming, debugging, frameworks, architectures. (K22-SEK2)	Y
How to interpret and implement a design, compliant with functional, non-functional and security requirements including principles and approaches to addressing legacy software development issues from a technical and socio-technical perspective. For example architectures, languages, operating systems, hardware, business change. (K24-SEK4)	Y
The factors affecting product quality and approaches for how to control them throughout the development process. For example security, code quality, coding standards. (K25)	
How to select and apply a range of software tools used in Software Engineering. (K26)	
Approaches to the interpretation and use of artefacts. For example UML, unit tests, architecture. (K27)	
Software Engineering Skills	

Identify and define software engineering problems that are non-routine and incompletely specified. (S16)	
Provide recommendations as to the most appropriate software engineering solution. (S17)	
Use appropriate analysis methods, approaches and techniques in software engineering projects to deliver an outcome that meets requirements. (S18)	
Implement software engineering projects using appropriate software engineering methods, approaches and techniques. (S19)	
Determine, refine, adapt and use appropriate software engineering methods, approaches and techniques to evaluate software engineering project outcomes. (S21-SES7)	Y